

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 28-MAY-2025

PAGE 1 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for life!

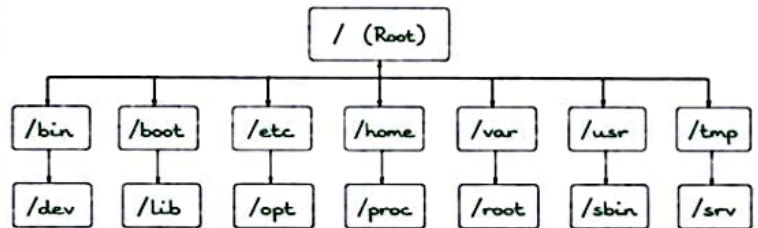
Keep Practicing !



1. What is a File System?

- A file system is the way Linux organizes and stores files on disk.
- It provides a structure to store, manage and retrieve data.
- Everything in Linux is a file. (even devices, directories, etc.)

2. File System Hierarchy (FHS)



FHS (Filesystem Hierarchy Standard) defines the directory structure in Linux.

3. Important Directories and Their Purpose

/	- Root directory. Top of the file system hierarchy.
/bin	- Essential user command binaries (ls, cp, mv, etc.)
/sbin	- System binaries for system administration (ifconfig, fdisk, etc.)
/etc	- Configuration files for system and applications.
/home	- Home directories for regular users.
/root	- Home directory for root user.
/var	- Variable data files (logs, spool, cache, etc.)
/tmp	- Temporary files. Data may be cleared on reboot.
/usr	- User programs and data (applications, libraries, docs).
/dev	- Device files (represents hardware devices).
/proc	- Virtual filesystem providing process and system information.
/boot	- Boot loader files and kernel images.
/lib	- Essential shared libraries and kernel modules.
/opt	- Optional/Third-party software packages.
/srv	- Data for services provided by this system.

4. Types of File Systems

- ext4 - Most common Linux file system.
- xfs - High performance & scalability.
- btrfs - Copy-on-write, snapshots, compression.
- tmpfs - Stores data in RAM.
- nfs - Network File System.



Tip: ext4 is default in most Linux distros.

5. Useful Commands (File System)

- ls - List files and directories
- cd - Change directory
- pwd - Print working directory
- tree - Display directory structure
- df -h - Disk space usage
- du -sh - Directory size
- mount - Mount a filesystem
- umount - Unmount a filesystem



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 29-MAY-2025

PAGE 2 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for life!

Keep Practicing !



6. File Types in Linux

- Regular file (text, binary, etc.)
- d Directory
- L Symbolic link
- c Character device file
- b Block device file
- p Named pipe (FIFO)
- s Socket

💡 Tip: Use command "ls -l" to see file types.

7. Absolute vs Relative Path

Absolute Path:

- Starts from root (/) and gives the complete path.
- Example: /home/user/docs/file.txt

Relative Path:

- Starts from current directory.
- Example: docs/file.txt or ../file.txt

💡 Tip: Use "pwd" to know current directory.

8. Navigate the File System

- pwd - Print working directory
- ls - List files and directories
- cd <dir> - Change directory
- cd .. - Go one level up
- cd / - Go to root directory
- cd ~ or cd - - Go to home directory

💡 Tip: Use "tab" key for auto-completion.

9. Listing Files (ls Command)

Command	Description
ls	List files in current directory
ls -l	Long format listing
ls -la	List all files (including hidden) in long format
ls -lh	Human readable file sizes
ls -R	Recursive listing (all subdirectories)
ls -t	Sort by modification time (newest first)
ls -S	Sort by file size (largest first)

10. File and Directory Operations

Command	Description	Example
touch <file>	Create an empty file	touch demo.txt
mkdir <dir>	Create a new directory	mkdir mydir
cp <src> <dest>	Copy file or directory	cp file.txt backup.txt
cp -r <src> <dest>	Copy directory recursively	cp -r mydir backupdir
mv <src> <dest>	Move or rename file/directory	mv old.txt new.txt
rm <file>	Remove (delete) file	rm file.txt
rm -r <dir>	Remove directory recursively	rm -r mydir

💡 Tip: Be careful with "rm -r". Deleted files are not easy to recover!

11. File Permissions (Basic)

Permission	Meaning	Example:
r (read)	- 4	-rw-r--r-- (644)
w (write)	- 2	6 4 4
x (execute)	- 1	Owner Group Others

💡 Tip: Use "chmod" to change permissions.

12. Disk Usage Commands

- df -h - Check disk space usage
- du -sh <dir> - Check size of directory
- du -h <file> - Check size of file
- df -i - Check inode usage

💡 Tip: Use "df -h" regularly to avoid disk full issues.

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 29-MAY-2025

PAGE 3 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for life!

Keep Practicing !



13. Hard Links vs Symbolic Links

Hard Link	Symbolic Link (Soft Link)
- Points to inode. - Same inode number for original file and hard link. - Cannot link across file systems. - Deleting original file does not affect hard link.	- Points to file path. - Different inode number. - Can link across file systems. - If original file is deleted, symlink becomes broken (dangling).

💡 Tip: Use "ln" for hard link and "ln -s" for symbolic link.

14. Inode in Linux

- Every file and directory has an inode.
- Inode stores metadata (permissions, owner, size, timestamps, etc.)
- Inode number is unique on a file system.
- Use "ls -li" to view inode number.

💡 Tip: Inode is like the index of a file.

15. Disk Usage Commands

df -h	- Check disk space usage (human readable)
du -sh <dir>	- Check size of a directory
du -ah <dir>	- Check size of files and directories inside
du -h --max-depth=1 <dir>	- Show size of immediate subdirectories
ncdu <dir>	- Interactive disk usage viewer

💡 Tip: Use "ncdu" for easily finding large files/directories.

16. Find Command Examples

find / -name file.txt	- Find file by name
find / -type d -name test	- Find directory by name
find / -size +100M	- Find files larger than 100MB
find / -mtime -7	- Modified in last 7 days
find / -user <username>	- Files owned by user
find / -perm 644	- Files with permission 644
find / -exec rm {} \;	- Execute command on result

💡 Tip: "find" is a powerful searching tool.

17. File System Types in Linux

File System	Description
ext4	- Default and most used in Linux.
xfs	- High performance, used for large systems.
btrfs	- Supports snapshots, compression, copy-on-write.
tmpfs	- Stored in memory (RAM).
nfs	- Network File System.

💡 Tip: ext4 is default in most popular distros.

18. Mount and Unmount

- Mount: Attach a filesystem to a directory.
- Unmount: Detach a filesystem.
- Mount command:
`mount <device> <mount-point>`
- Unmount command:
`umount <mount-point>`
- Check mounted filesystems:
`mount | column -t`

💡 Tip: You need root/sudo privileges to mount/unmount.

19. /etc/fstab File

- /etc/fstab contains information about filesystems that should be mounted automatically at boot.
- Format:

<device>	<mount_point>	<type>	<options>	<dump>	<pass>
----------	---------------	--------	-----------	--------	--------

- After editing, run: `mount -a` (to apply changes)

💡 Tip: Be careful while editing /etc/fstab.

20. Best Practices

- ✓ Keep file system organized.
- ✓ Use meaningful names for files and directories.
- ✓ Regularly clean temporary files.
- ✓ Monitor disk usage.
- ✓ Use appropriate permissions.
- ✓ Backup important data.
- ✓ Use symbolic links carefully.
- ✓ Avoid deleting system files.



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 29-MAY-2025

PAGE 4 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



21. Wildcards (Globbing)

Wildcard	Meaning
*	Matches any number of characters (including zero)
?	Matches any single character
[abc]	Matches any one character in the set
[a-z]	Matches any character in the range
[!abc]	Matches any one character NOT in the set

💡 Tip: Wildcards are used with `ls`, `cp`, `mv`, `rm`, etc.

22. Redirection and Pipes

Redirection

> Overwrite output to a file
>> Append output to a file
2> Redirect error output
&> Redirect both output and error

Pipes

| Pass output of one command as input to another

Examples:

```
ls -l > list.txt           # write output to file
echo "Hello" >> msg.txt    # append to file
ls /no_such 2> error.log  # redirect errors
df -h | grep sda          # pipe output to grep
```

💡 Tip: Pipes (|) are very powerful for chaining commands.

23. Hidden Files and Directories

- In Linux, files and directories starting with `.` are hidden.
- Use `-a` option with `ls` to see hidden files.

Examples:

```
ls -a           # show all files including hidden
ls -la          # detailed list including hidden
touch .hiddenfile # create a hidden file
```

💡 Tip: `.` (dot) means current directory.

24. File Ownership and Permissions

ls -l output example:

-rwxr-xr--	1	user	group	1024	May 29 10:00	script.sh
Permissions	Links	Owner	Group	Size	Date	Name

Permission Types:

r - Read (4)
w - Write (2)
x - Execute (1)
- - No permission (0)

Who gets the permissions?

u - User (owner)
g - Group
o - Others
a - All (u + g + o)

💡 Tip: Permission example: `rw-r--r--` means 644.

25. Change Permissions (chmod)

Numeric (Octal) Method

Digit	Permission	Digit	Permission
4	Read (r)	2	Read (r)
2	Write (w)	1	Execute (x)

Examples:

```
chmod 755 script.sh # rwxr-xr-x
chmod 644 file.txt  # rw-r--r--
chmod 600 secret.txt # rw-----
```

💡 Tip: Add numbers (r=4, w=2, x=1) for each (u,g,o).

26. Change Ownership (chown)

Change file/directory owner:

```
chown newowner file.txt
```

Change owner and group:

```
chown newowner:newgroup file.txt
```

Change group only:

```
chgrp newgroup file.txt
```

💡 Tip: Requires root or sudo privileges.

27. Disk Usage Commands

Command	Description
<code>df -h</code>	Show disk space usage of filesystems
<code>du -sh <dir></code>	Show size of a directory (summary)
<code>du -h --max-depth=1 <dir></code>	Show size of subdirectories (one level)
<code>ncdu <dir></code>	Interactive disk usage viewer (if installed)

💡 Tip: Use `du` to find large directories and save space.

28. Clean Up Unnecessary Files

- ✓ Remove temporary files in `/tmp`
- ✓ Use `rm` to delete unused files
- ✓ Use `du` to identify large files/directories
- ✓ Clear package cache: `sudo apt clean` (Debian/Ubuntu)
- ✓ Clear logs if old: `sudo journalctl --vacuum-time=7d`
- ✓ Keep your system clean and optimized!



💡 Tip: Clean system = Better performance!

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 29-MAY-2025

PAGE 5 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



29. Directory Structure Best Practices

- Keep directories organized and meaningful.
- Group related files together.
- Avoid deep nesting.
- Use lowercase letters.
- Use hyphens instead of spaces.
- Follow standard locations.

Tip: A good structure = Easy to find and manage.

30. Logs Location in Linux

Log File / Directory	Description
/var/log/syslog	General system logs (Debian/Ubuntu)
/var/log/messages	General system logs (RHEL/CentOS)
/var/log/auth.log	Authentication logs (Debian/Ubuntu)
/var/log/secure	Authentication logs (RHEL/CentOS)
/var/log/dmesg	Kernel ring buffer messages
/var/log/kern.log	Kernel related logs
/var/log/boot.log	Boot loader logs
/var/log/lastlog	Last login information

Tip: Logs are very useful for troubleshooting.

31. Disk Usage and df Command

`df -h` → Human readable disk usage

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	50G	20G	28G	42%	/
tmpfs	1.9G	0	1.9G	0%	/dev/shm

- Size - Total size
- Used - Used space
- Avail - Available space
- Use% - Usage percentage
- Mounted on - Mount point

Tip: Use `df -h` to check disk space usage.

32. du Command (Disk Usage)

`du -sh <directory>` → Summary of directory size

Command	Description
<code>du -sh dir/</code>	Summary size of directory
<code>du -sh *</code>	Summary size of all in current dir
<code>du -h --max-depth=1</code>	Size of subdirectories (one level)
<code>du -ah</code>	Size of all files and subdirectories
<code>du -csh dir/</code>	Total size with summary

Tip: `du` helps to find large files/directories.

33. Find Large Files

Find files larger than 100MB

`find / -type f -size +100M`

Find files modified in last 7 days

`find / -type f -mtime -7`

Find and delete files older than 30 days

`find /tmp -type f -mtime +30 -delete`

Tip: Be careful with "-delete" option!

34. Working with Links

Hard Link

- > Points to same inode.
- > Cannot cross filesystem.
- > Original file must exist.
- > Inode count increases.

`ln file.txt hardlink.txt`

Symbolic Link (Soft Link)

- > Points to file path.
- > Can cross filesystems.
- > Works even if original is deleted (broken link).

`ln -s file.txt softlink.txt`

Tip: Use `ls -li` to see inode numbers.

35. Backup Important Directories

Using `tar` command

`tar -czvf backup.tar.gz /home/user/docs`

Options:

- c → Create archive
- z → Compress using gzip
- v → Verbose
- f → File name



Tip: Regular backups save your time and data.

36. Restore from Backup

Extract `tar.gz` file

`tar -xzf backup.tar.gz`

Extract to specific directory

`tar -xzf backup.tar.gz -C /restore/path`

List contents without extracting

`tar -tzvf backup.tar.gz`

Tip: Test your backup by restoring it!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 29-MAY-2025

PAGE 6 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



37. Special Directories

Directory	Purpose
/root	Home directory of root user
/tmp	Temporary files (cleared on reboot)
/proc	Virtual filesystem (process + kernel info)
/sys	Sysfs filesystem (kernel devices info)
/dev	Device files
/run	Runtime data (since boot)

💡 Tip: Know these directories very well!

38. Environment Variables

- Variables that store system or user info.
- Use `env` or `printenv` to see all variables.
- Common variables:
 - `PATH` : Command search path
 - `HOME` : Home directory
 - `USER` : Current username
 - `SHELL` : Current shell
 - `PWD` : Current working directory

💡 Tip: Use `echo $VARIABLE_NAME` to see value.

39. PATH Variable Example

- `PATH` tells Linux where to look for commands.
- Directories in `PATH` are separated by colon (:).
- Example:

```
echo $PATH
```

```
/usr/local/sbin : /usr/local/bin : /usr/sbin : /usr/bin : /sbin : /bin
```

High Priority ← Search Order → Low Priority

💡 Tip: Add custom path: `export PATH=$PATH:/opt/mybin`

40. Aliases

- Alias is a shortcut for a command.
- Use `alias` to create and `alias` to view.
- Example:

```
alias ll='ls -aF'
alias gs='git status'
alias ..='cd ..'
```

💡 Tip: Aliases are user specific (in `~/.bashrc`).

41. File Name Wildcards

Wildcard	Example	Matches
*	*.txt	All .txt files
?	file?.log	file1.log, file2.log
[abc]	file[abc].txt	filea.txt, fileb.txt, filec.txt
[a-z]	file[a-z].txt	filea.txt to filez.txt
[!abc]	file[!abc].txt	Any except a, b or c

💡 Tip: Wildcards save time in listing and managing files.

42. Links in Linux

- Hard Link: Another name for the same inode.
 - Cannot cross filesystem.
- Soft Link (Symbolic Link): Points to file path.
 - Can cross filesystem.
 - If original deleted, link is broken.

Type	Create Command	Example
Hard Link	<code>ln source.txt hardlink.txt</code>	Same inode
Soft Link	<code>ln -s source.txt softlink.txt</code>	Different inode

💡 Tip: Use `ls -li` to see inode numbers.

43. Check File Type

- Use `file` command to know file type.
- | | |
|-------------------------------|--|
| <code>file /etc/passwd</code> | Output: <code>/etc/passwd: ASCII text</code> |
| <code>file script.sh</code> | <code>script.sh : Bourne-Again shell script</code> |
| <code>file image.png</code> | <code>image.png : PNG image data</code> |

💡 Tip: Useful to identify unknown file types.

44. Text Processing Basics

- `cat file.txt` → Show file content
- `less file.txt` → View with scroll
- `head -n 10 file.txt` → First 10 lines
- `tail -n 10 file.txt` → Last 10 lines
- `grep "pattern" file.txt` → Search pattern
- `wc -l file.txt` → Count lines
- `sort file.txt` → Sort lines
- `uniq file.txt` → Remove duplicate lines

💡 Tip: Combine tools using `|` (pipe) for powerful results.

45. Redirection Summary

Symbol	Meaning	Example
<code>></code>	Redirect output (overwrite)	<code>ls > list.txt</code>
<code>>></code>	Redirect output (append)	<code>echo "Hi" >> msg.txt</code>
<code>2></code>	Redirect error output	<code>cmd 2> error.log</code>
<code>&></code>	Redirect both output & error	<code>cmd &> all.log</code>
<code> </code>	Pipe output to another command	<code>ls grep txt</code>

💡 Tip: Redirection + pipes = automation power!

46. Quick Revision Checklist

- ✓ Understand Linux directory hierarchy
- ✓ Use basic commands (`ls`, `cd`, `pwd`, `cp`, `mv`, `rm`, `touch`)
- ✓ Work with file permissions and ownership
- ✓ Use wildcards and redirection
- ✓ Check disk usage and logs
- ✓ Create and manage links
- ✓ Practice daily - that's the key!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 30-MAY-2025

PAGE 7 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



47. Hard Link vs Soft Link (Summary)

Feature	Hard Link	Soft Link (Symbolic Link)
Inode	Same inode	Different inode
Cross Filesystem	Not allowed	Allowed
Original deleted	Data still exists	Link broken
Type	File or Directory	File or Directory
Command	<code>ln</code>	<code>ln -s</code>

💡 Tip: Hard link = Backup of inode, Soft link = Shortcut.

48. Using Tabs for Auto Completion

- Press Tab key to auto-complete commands, file names and directories.

Example:

```
ls Doc<Tab>    → ls Documents/  
cd /var/L<Tab> → cd /var/log/
```

💡 Tip: Saves time and reduces typing errors.

49. File System Hierarchy (FHS)

- Linux follows a standard directory structure called FHS.

Directory	Purpose
<code>/</code>	Root directory of entire system
<code>/bin</code>	Essential user command binaries
<code>/sbin</code>	System binaries (admin commands)
<code>/etc</code>	System configuration files
<code>/home</code>	User home directories
<code>/var</code>	Variable data like logs, spool, cache
<code>/usr</code>	User programs and data
<code>/tmp</code>	Temporary files
<code>/opt</code>	Optional third-party software

💡 Tip: FHS brings consistency across Linux distributions.

50. Mount Points

- A mount point is a directory where a filesystem is attached.
- Use `mount` to attach and `umount` to detach.

Example:

```
# Mount a device  
mount /dev/sdb1 /mnt/mydisk  
  
# Unmount  
umount /mnt/mydisk
```

💡 Tip: Always unmount before removing a device.

51. Disk Usage Summary

Command	Description
<code>df -h</code>	Summary of disk usage
<code>du -sh <dir></code>	Size of a directory
<code>du -ah <dir> less</code>	Detailed size with scroll
<code>ncdu <dir></code>	Interactive disk usage viewer

💡 Tip: Use `du` and `ncdu` to find space hogs.

52. Important Points to Remember

- ✓ Everything in Linux is a file.
- ✓ Know the purpose of each directory.
- ✓ Use proper permissions for security.
- ✓ Use links wisely.
- ✓ Monitor disk usage regularly.
- ✓ Back up important data.
- ✓ Practice commands daily!



53. Common Workflow Example

1. Go to home	<code>cd ~</code>	# Go to home directory
2. Create folder	<code>mkdir projects</code>	
3. Enter folder	<code>cd projects</code>	
4. Create file	<code>touch notes.txt</code>	
5. Write data	<code>echo "Hello Linux" > notes.txt</code>	
6. View file	<code>cat notes.txt</code>	
7. Check size	<code>ls -lh notes.txt</code>	
8. Make link	<code>ln -s notes.txt link.txt</code>	
9. List files	<code>ls -l</code>	
10. Disk usage	<code>df -h</code>	

💡 Tip: Practice this workflow in your system.

54. Mini Cheat Sheet

- `pwd` - Show current directory
- `ls -l` - List files in long format
- `ls -a` - Show hidden files
- `cd ..` - Go one level up
- `cd /` - Go to root
- `mkdir <dir>` - Create directory
- `touch <file>` - Create file
- `cp <src> <dst>` - Copy file
- `mv <src> <dst>` - Move or rename
- `rm <file>` - Remove file
- `rm -r <dir>` - Remove directory
- `ln <src> <dst>` - Create hard link
- `ln -s <src> <dst>` - Create soft link
- `df -h` - Disk usage summary

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 30-MAY-2025

PAGE 8 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



1. Linux File System Hierarchy

- Linux follows a single, unified directory structure
- Everything is a file
- The root directory is /.
- All other directories are under it
- Directories are used to organize files

Common Directories

/bin	- Essential binaries
/etc	- Configuration files
/home	- User home directories
/var	- Variable data (logs, cache, etc.)
/tmp	- Temporary files
/usr	- User programs and files
/dev	- Device files
/proc	- Process information

💡 Tip: Think of / as the top of a tree

2. Absolute vs Relative Paths

Absolute Path:

- Starts from the root /
- Example: /home/user/file.txt

Relative Path:

- Starts from the current directory
- Example: docs/file.txt

Useful Commands

pwd	- Print working directory
cd	- Change directory
ls	- List files

💡 Tip: Absolute paths are unambiguous

3. File Types

- Regular File - Contains data
- Directory - Contains files and other directories
- Symbolic Link - Shortcut to another file
- Hard Link - Multiple names for the same file
- Device File - Represents devices
- FIFO - Named pipe for inter-process communication

Examples

-rwxr-xr-x	regular file
drwxr-xr-x	directory
lrwxrwxrwx	symbolic link

💡 Tip: Use ls -l to see file types

4. Permissions

Each file has permissions for:

Owner	Group	Others
Read (r)	Write (w)	Execute (x)

Numeric Representation

r = 4 w = 2 x = 1

Example: rwx r-x r-- = 754

Commands

chmod	- Change permissions
chown	- Change owner
chgrp	- Change group

💡 Tip: Permissions control access to files

5. Viewing File Contents

cat	- Display file content
less	- View content one page at a time
head	- Show first 10 lines
tail	- Show last 10 lines
nl	- Show lines with numbers

Example

cat /etc/os-release

💡 Tip: Use less for long files

6. Creating and Deleting Files

Create File: touch filename

Delete File: rm filename

Delete Directory: rm -r directory

Delete Non-empty Directory:
rm -rf directory

Example

touch notes.txt
rm notes.txt

💡 Tip: Always double-check before deleting

7. Moving and Copying

Move or Rename: mv old new

Copy File: cp file1 file2

Copy Directory: cp -r dir1 dir2

Example

mv file.txt /home/user/
cp -r docs/ backup/

💡 Tip: cp -r is required for directories

8. Wildcards

Wildcards help match multiple files.

- Matches any number of characters
- ? Matches any single character
- [] Matches any character in brackets

Example

ls *.txt # all txt files
ls file?.log # file1.log, file2.log
ls [a-z]* # files starting with a-z

💡 Tip: Wildcards make file management easy

9. Environment Variables

Environment variables store system and user information

View All: env

View One: echo \$HOME

Set Variable: export VAR=value

Example

export MYVAR="Hello DevOps"
echo \$MYVAR

💡 Tip: Variables are case sensitive

10. Standard Streams

Linux uses three standard streams:

- 0 - Standard Input (stdin)
- 1 - Standard Output (stdout)
- 2 - Standard Error (stderr)

Example

ls nonexistent.txt 2> error.log

💡 Tip: 2> redirects errors to a file.

11. Special Characters

Some special characters used in Linux:

- ~ Home directory
- . Current directory
- .. Parent directory
- > Redirect output (overwrite)
- >> Redirect output (append)
- | Pipe output to another command

💡 Tip: These characters are powerful!

12. Practical Examples

1. List all files including hidden
ls -la
2. Show disk usage
df -h
3. Find files by name
find / -name "*.conf"
4. Check file type
file /bin/ls

💡 Tip: Practice these commands daily!

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 31-MAY-2025

PAGE 9 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



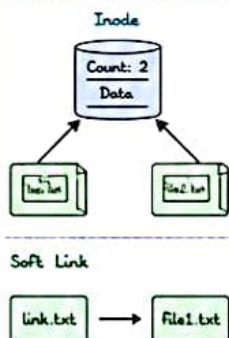
13. Links Deep Dive

Hard Link

- Points to same inode.
- Cannot cross filesystem.
- Count increases.
- Deleting original doesn't affect the link.

Soft Link (Symbolic Link)

- Points to file path.
- Can cross filesystem.
- If original is deleted, link becomes broken.



💡 Tip: Use hard link for backups, soft link for shortcuts.

14. Hidden Files

- Files starting with dot (.) are hidden.
- Used for config and settings.
- Often ignored in normal listings.

Example

```
ls -a      # show all files
ls -la     # show with details
```

💡 Tip: Don't edit hidden files unless you know what you're doing.

15. File Ownership

- Every file has an owner and group.
- Owner has full control by default.
- Change owner using `chown`.
- Change group using `chgrp`.

Example

```
chown user file.txt      # change owner
chgrp dev file.txt       # change group
chown user:dev file.txt  # owner:group
```

💡 Tip: Ownership defines responsibility.

16. Understanding Permissions

Permissions are shown as 3 sets (rwx).

r = read (4) w = write (2) x = execute (1)

User	Group	Others	Numeric
rwX	rwX	rwX	777
rw-	r--	r--	644
r-X	r-X	r-X	555
r--	---	---	400

💡 Tip: 777 is risky. Use least privilege.

17. Changing Permissions

- Use `chmod` to change permissions.
- Two methods: Symbolic and Numeric.

Symbolic Example

```
chmod u+x script.sh  # add execute to user
chmod go-w file.txt  # remove write from group & others
```

Numeric Example

```
chmod 755 script.sh  # rwxr-xr-x
chmod 640 file.txt   # rw-r-----
```

💡 Tip: Use `chmod -R` to change recursively.

18. Sticky Bit

- Special permission on directories.
- Only owner can delete files in that dir.
- Prevents accidental deletion.

Example

```
chmod +t /tmp  # set sticky bit
ls -ld /tmp    # verify
```

💡 Tip: /tmp uses sticky bit by default.

19. Default Permissions (umask)

- `umask` sets default permissions for newly created files and directories.
- Files: 666 - `umask`
- Directories: 777 - `umask`

Example

```
umask      # show current mask
umask 022  # set mask
```

💡 Tip: `umask 022` is a good default.

20. Disk Quota

- Limits disk usage for users.
- Prevents disk from filling up.
- Set by admin using quota tools.

Check Quota

```
quota -s      # summary
quota -u user  # user quota
edquota -u user # edit quota
```

💡 Tip: Quotas help in shared environments.

21. File Attributes

- Use `chattr` to change file attributes.
- i = immutable (cannot modify)
- a = append only
- h = hide file

Example

```
chattr +i important.txt # immutable
chattr -i important.txt  # remove immutable
```

💡 Tip: Useful for protecting important files.

22. Finding Files

- `find` is a powerful tool to search files.
- Search by name, type, size, date.

Examples

```
find / -name file.txt  # search by name
find /home -type d     # search directories
find / -size +100M     # larger than 100MB
find / -mtime -7       # modified in last 7 days
```

💡 Tip: `find` is better than `locate` for real-time search.

23. Locating Commands

- `which` shows path of executable.
- `whereis` shows binary, source and man page.
- `locate` is faster but uses updatedb.

Example

```
which ls      # /bin/ls
whereis ls    # ls: /bin/ls /usr/share/man/...
locate nginx.conf # search in db
```

💡 Tip: Run `updatedb` for updated results.

24. Inode Information

- Every file has an inode number.
- Use `ls -li` to view inode.
- Helps in troubleshooting.

Example

```
ls -li file.txt  # show inode
df -li          # inode usage
```

💡 Tip: Inodes are limited, not just disk space.

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 01-JUNE-2025

PAGE 10 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



25. Special Files

- Special Files represent devices or system resources.

Types:

- Character Device - /dev/null
- Block Device - /dev/sda
- Named Pipe (FIFO) - /tmp/myfifo
- Socket - Used in IPC

💡 Tip: Special Files are in /dev directory.

26. /proc File System

- Virtual filesystem providing process and system information.
- Data is in memory, not on disk.

Example

```
cat /proc/cpuinfo # CPU info
cat /proc/meminfo # Memory info
ls /proc/<pid>     # Process info
```

💡 Tip: Very useful for monitoring and debugging.

27. /sys File System

- Provides information about devices, drivers and kernel.
- Interface to kernel data structures.

Example

```
ls /sys/class      # Device classes
cat /sys/kernel/hostname # Hostname
```

💡 Tip: Used by system tools and udev.

28. tmpfs and RAM Disks

- tmpfs stores data in RAM.
- Faster but data is lost on reboot.

Example

```
mount -t tmpfs tmpfs /mnt/ramdisk
```

💡 Tip: Used for temporary or performance sensitive data.

29. Bind Mounts

- Makes a file or directory appear in another location in filesystem.

Example

```
mount --bind /source/path /target/path
# or
mount -o bind /source /target
```

💡 Tip: Useful for backups and containers.

30. Loopback Devices

- Treat a file as a block device.
- Used for mounting disk images.

Example

```
losetup /dev/loop0 /path/to/image.iso
mount /dev/loop0 /mnt/iso
```

💡 Tip: Common in testing and recovery.

31. Journaling File Systems

- Keep a journal of changes before writing.
- Ensures faster recovery after crash.

Common Types:

ext4, xfs, btrfs

💡 Tip: ext4 is default in most Linux distros.

32. fsck - File System Check

- Checks and repairs filesystem errors.
- Useful after unclean shutdown.

Example

```
fsck /dev/sda1 # check only
fsck -y /dev/sda1 # auto repair
```

💡 Tip: Run on unmounted filesystem.

33. du and df Commands

df - Disk Free

df -h # Human readable

df -Th # Show filesystem type

du - Disk Usage

du -sh /home/user # Summary

du -h --max-depth=1 /var # Detailed

💡 Tip: df shows free space, du shows usage.

34. Inotify

- Monitors filesystem events (create, modify, delete).
- Used by backup tools, editors, and sync tools.

Example

```
inotifywait -m /path/to/dir
# -m : monitor continuously
```

💡 Tip: Great for real-time file monitoring.

35. File Command

- Identifies file type.
- Useful to detect binary, text, script, etc.

Example

```
file /bin/ls # binary
file notes.txt # ASCII text
```

💡 Tip: Don't rely on extension, use file!

36. stat Command

- Shows detailed information about a file.
- Includes permissions, size, timestamps, etc.

Example

```
stat notes.txt
```

💡 Tip: Useful for troubleshooting and audit.

37. Summary

- ✓ Linux has a strict directory hierarchy.
- ✓ Everything is a file - even devices.
- ✓ Permissions control access.
- ✓ Hard links and soft links are different.
- ✓ Use the right tool for the right job.

- ✓ Practice commands daily.
- ✓ Understand and explore your system.
- ✓ This foundation is key for DevOps success!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 02-JUNE-2025

PAGE 11 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



38. File Permissions (Octal)

- 3 digits represent user, group, others.
- Each digit = rwx in binary.

Example

Octal	Binary	Permission
7	111	rwx
6	110	rw-
5	101	r-x
4	100	r--
0	000	---

Tip: Common permission - 755 for executables

39. File System Types

- ext4 - Default, journaling, reliable.
- XFS - High performance, large files.
- NTFS - Windows native file system.
- FAT32 - Compatible with many OS.
- tmpfs - In-memory file system.

Example

```
lsblk # List file systems
df -T # Show file system types
blkid # Show filesystem info
```

Tip: Choose file system based on use case.

40. Mount Options

- Controls how a filesystem is mounted.
- Common options:

```
ro - Read-only
rw - Read-write
noexec - Disallow execution
nosuid - Ignore setuid/setgid
nodev - Ignore device files
```

Example

```
mount -o rw,noexec /dev/sdb1 /mnt/data
```

Tip: Use secure options for production.

41. Device Files

- Represent devices in /dev directory
- Block devices - Store data in blocks.
- Character devices - Stream of data.

Example

```
ls -l /dev/sda # Block device
ls -l /dev/tty # Character device
mknod mychar c 1 7 # Create char device
```

Tip: /dev is the interface to hardware.

42. File Descriptors

- Integer handles for open files.
- Standard descriptors:
 - 0 - stdin (standard input)
 - 1 - stdout (standard output)
 - 2 - stderr (standard error)

Example

```
ls -l /proc/$$/fd # Current process FDs
```

Tip: Every process handles files via FDs

43. /proc File System

- Virtual filesystem with process and kernel info.
- Provides real-time system information.

Example

```
ls /proc # List all processes
cat /proc/cpuinfo # CPU information
cat /proc/meminfo # Memory information
```

Tip: Useful for monitoring and debugging.

44. Understanding Inodes

- Inode stores metadata of a file.
- Contains: permissions, owner, size, timestamps, and block pointers.

Example

```
ls -li # Shows inode number
```

Tip: Inodes identify files, not names.

45. Hard Links vs Soft Links

- Hard Link - Points to same inode.
- Soft Link - Points to another file path.

Hard Link	Soft Link
Same inode	Different inode
Cannot cross filesystem	Can cross filesystem
Cannot link directories	Can link directories

Example

```
ln file1 file1_hard # Hard link
ln -s file1 file1_soft # Soft link
```

Tip: Use soft links for flexibility.

46. File System Check

- Check and repair filesystem errors.
- On unclean shutdown, run fsck.

Example

```
fsck /dev/sda1 # Check filesystem
fsck -y /dev/sda1 # Auto repair
```

Tip: Run fsck on unmounted filesystem.

47. Disk Quota

- Limits disk usage for users or groups.
- Prevents overuse of disk space.

Example

```
quota -s # Summary
quota -u user # User quota
edquota -u user # Edit quota
```

Tip: Quotas help manage shared environments

48. File Compressions

- Compress files to save space.
- Common tool: gzip

Example

```
gzip file.txt # Compress
gunzip file.txt.gz # Decompress
tar -czf files.tar.gz dir/ # Archive + Compress
```

Tip: Use compression for backups and transfer

49. Log Rotation

- Prevent logs from growing indefinitely.
- Uses logrotate to manage logs.

Example

```
logrotate /etc/logrotate.conf # Manual run
logrotate -f /etc/logrotate.conf # Force run
```

Tip: Keeps logs manageable and safe.

50. System Time & Time Zone

- System time affects logs and cron jobs.
- Set time and timezone.

Example

```
date # Show current time
timedatectl # Show time status
timedatectl set-timezone Asia/Kolkata
```

Tip: Keep time correct for consistency

51. Additional Commands

```
uname -a # System info
uptime # System uptime
whoami # Current user
id # User and group ID
df -h # Disk usage (human)
```

```
du -sh dir # Directory size
free -h # Memory usage
ps aux # List all processes
top # Real-time process view
which command # Command location
```

Tip: Learn and practice these daily

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 03-JUNE-2025

PAGE 12 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



52. Environment Variables

- Store system-wide or user-specific configuration.
- Used by system and applications.

Example

```
echo $HOME      # Home directory
echo $PATH       # Executable search path
printenv         # Show all variables
```

💡 Tip: Environment variables control behavior.

53. Path & \$PATH

- \$PATH tells system where to find executable commands.
- Directories in \$PATH are searched in order.

Example

```
echo $PATH
export PATH=$PATH:/my/tools/bin
which ls        # Shows full path of command
```

💡 Tip: Add custom scripts to \$PATH.

54. Wildcards (Globbing)

- Used to match multiple filenames.
- Makes working with files easier.

Common Wildcards

```
*      # Matches any number of characters
?      # Matches any single character
[abc]  # Matches any one of a, b or c
[a-z]  # Matches any letter a to z
```

Example

```
ls *.txt      # All txt files
ls file?.log  # file1.log, file2.log
ls [a-c]*     # Starts with a, b or c
```

💡 Tip: Wildcards save time and effort.

55. Finding Files

- Find command searches files in directory hierarchy.
- Powerful with filters.

Example

```
find / -name file.txt
find /home -type d -name logs
find . -size +100M    # > 100MB
find . -mtime -7      # Modified in last 7 days
```

💡 Tip: Use find for advanced searches.

56. Locating Commands

- which shows path of executable.
- whereis gives binary, source, and man page locations.

Example

```
which python
whereis ls
type cd      # Built-in or external
```

💡 Tip: which is more accurate for executables.

57. File Archiving

- Archive multiple files into single file.
- Saves space and easy to transfer.

Example

```
tar -czf backup.tar.gz /home/user
tar -xzf backup.tar.gz      # Extract
tar -cvf files.tar file1 file2
```

💡 Tip: tar is most used in Linux.

58. Redirection

- Redirect output or input of commands.
- Useful for logging and automation.

Types

```
>      # Redirect output (overwrite)
>>     # Redirect output (append)
<      # Redirect input
2>     # Redirect error
2>&1    # Redirect error to output
```

Example

```
ls -l > list.txt
cat file.txt >> list.txt
grep error log.txt 2> errors.log
command > all.log 2>&1
```

💡 Tip: Redirection helps in logs and reports.

59. Pipes (|)

- Pass output of one command as input to another.
- Combine commands together.

Example

```
ps aux | grep nginx
ls -l | less
df -h | sort -h
cat file.txt | wc -l # Count lines
```

💡 Tip: Pipes make commands more powerful.

60. Command Chaining

- Run multiple commands in sequence.
- Use ; , && , || for control.

Operators

```
;      # Run always
&&     # Run next only if previous succeeds
||     # Run next only if previous fails
```

Example

```
mkdir test ; cd test ; touch a.txt
command1 && echo "Success"
command1 || echo "Failed"
```

💡 Tip: Useful in scripting and automation.

61. Crontab (Scheduling Jobs)

- Schedule tasks to run automatically.
- Crontab uses time based triggers.

Example

```
crontab -e      # Edit crontab
crontab -l      # List jobs
crontab -r      # Remove all jobs
```

Crontab Format

```
* * * * * command-to-run
Minute Hour Day Month Weekday
```

💡 Tip: Great for regular backups and monitoring.

62. Disk Usage Summary

- Check disk usage and find large files.
- Keep system clean and fast.

Commands

```
df -h          # Disk usage
du -sh *       # Folder sizes
du -ah /home | sort -rh | head -10
ncdu /home     # Interactive disk usage
```

💡 Tip: Monitor disk before it gets full.

63. Summary of File Commands

Command	Purpose
ls	List files and directories
cd	Change directory
pwd	Show current directory
mkdir	Create directory
cp	Copy files
mv	Move or rename
rm	Remove files
chmod	Change permissions
find	Search files
tar	Archive files

💡 Tip: Practice daily to master commands.

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 03-JUNE-2025

PAGE 13 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



64. Space in Filenames

- Spaces in filenames are allowed.
- Use quotes or escape with backslash.

Example

```
touch "my file.txt"    # Using quotes
touch my\ file.txt     # Using \
```

💡 Tip: Quotes are safer for spaces.

65. Case Sensitivity

- Linux is case-sensitive.
- File, file, FILE are different.

Example

```
touch file.txt
touch file.txt } # 3 different files
touch FILE.txt
```

💡 Tip: Be careful with uppercase & lowercase.

66. Home Directory

- Each user has a home directory.
- Shown by ~ or \$HOME.

Example

```
cd ~          # Go to home
echo $HOME    # Show home path
```

💡 Tip: ~ is shortcut for your home.

67. Temporary Files

- Store temporary files in /tmp.
- Files may be deleted on reboot.

Example

```
touch /tmp/tmpfile.txt
ls /tmp          # List temp files
```

💡 Tip: Don't store important data in /tmp.

68. Symbolic vs Hard Links

Feature	Hard Link	Symbolic Link
Inode	Same	Different
Cross FS	No	Yes
Original delete	Still exists	Broken link

Example

```
ln file.txt hardlink.txt # Hard link
ln -s file.txt symlink.txt # Soft link
```

💡 Tip: Use hard link for backup, soft link for shortcuts.

69. Copy vs Move

- cp copies files or directories.
- mv moves or renames files.

Example

```
cp file.txt file_copy.txt
mv oldname.txt newname.txt
```

💡 Tip: mv can be undo only with backups.

70. Remove Files & Directories

- rm removes files.
- rmdir removes empty directories.
- rm -r removes directory with content.

Example

```
rm file.txt      # Remove file
rmdir emptydir   # Remove empty dir
rm -r mydir      # Remove dir + content
```

💡 Tip: rm -r is permanent, use carefully.

71. Tree Command

- Shows directory structure in tree view.
- Very useful to understand hierarchy.

Example

```
tree            # Show current dir tree
tree /etc       # Show /etc tree
```

💡 Tip: Install if not available: sudo apt install tree

72. File Type

- Linux files are of different types.
- Run: file filename

Example

```
file /etc/passwd
file myfile.txt
```

💡 Tip: Helps to identify file type and format.

73. Watch Command

- Repeats a command at regular intervals.
- Useful for monitoring.

Example

```
watch -n 2 df -h    # Every 2 sec
watch -n 5 ls -l     # Every 5 sec
```

💡 Tip: Press Ctrl+C to stop watch

74. Sort & Unique

- sort sorts lines in a file.
- uniq removes duplicate lines.

Example

```
sort names.txt      # Sort lines
sort -r names.txt   # Reverse sort
sort names.txt | uniq # Unique lines
```

💡 Tip: Great for reports and logs.

75. Practice Commands

1. Create file: touch practice.txt
2. Write to file: echo Hello > practice.txt
3. Append data: echo World >> practice.txt
4. View file: cat practice.txt
5. Count lines: wc -l practice.txt
6. Copy file: cp practice.txt copy.txt
7. Move file: mv copy.txt moved.txt
8. Delete file: rm moved.txt

💡 Tip: Practice daily, become stronger!



DAY 4 SUMMARY

- ✓ Understood file system structure
- ✓ Worked with files and directories
- ✓ Learned permissions and ownership

- ✓ Used important file system commands
- ✓ Practiced good Linux habits

KEEP PRACTICING - YOU ARE DOING GREAT!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 04-JUNE-2025

PAGE 14 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



76. Hidden Files

- Files starting with '.' are hidden.
- Used for config and system purpose.

Example

```
ls -a          # Show all including hidden
.bashrc      .profile  .gitconfig
```

💡 Tip: Hidden files keep your system settings.

77. File Ownership

- Every file has an owner and a group.
- Owner can change permissions.

Example

```
ls -l file.txt          # Shows owner & group
-rw-r--r-- 1 user1 devops 1024 Jun 4 File.txt
           owner  group
```

💡 Tip: Know who owns the file.

78. Change Ownership (chown)

- Change file or directory owner.
- Syntax: chown user:group file

Example

```
chown user1 file.txt
chown user1:devops file.txt
```

💡 Tip: Use with care. Wrong owner blocks access.

79. Change Group (chgrp)

- Change group ownership only.
- Syntax: chgrp group file

Example

```
chgrp devops file.txt
```

💡 Tip: Helpful in team environments.

80. File Permissions (Basics)

- 3 types: read (r), write (w), execute (x)
- For owner, group, others.

Example

```
chmod 755 script.sh # rwxr-xr-x
chmod 644 data.txt  # rw-r--r--
```

💡 Tip: 7 = 4+2+1 (rwx) 6 = 4+2 (rw)

81. Permission Number Meaning

Number	Permission	Meaning
4	r	read
2	w	write
1	x	execute
0	-	no permission

💡 Tip: Add numbers to set permissions.

82. Default Permissions

- Files: 666 → rw-rw-rw- (before umask)
- Directories: 777 → rwxrwxrwx (before umask)

Example

```
umask          # Show current umask
umask 022      # Set umask
```

💡 Tip: umask decides default permissions.

83. Sticky Bit

- Set on directories to restrict deletion.
- Only owner can delete files.

Example

```
chmod +t /tmp # Set sticky bit
ls -ld /tmp   # Verify
```

💡 Tip: /tmp uses sticky bit for safety.

84. SUID, SGID, Sticky Bit

bit	Purpose	applies to
SUID	Run as file owner	Files
SGID	Run as group owner	Files/Dirs
Sticky	Restrict deletion	Directories

💡 Tip: Useful for security and control.

85. Links in Linux

- Hard Link: Points to same inode.
- Soft Link (Symlink): Points to file path.

Example

```
ln file.txt hardlink.txt # Hard link
ln -s file.txt symlink.txt # Soft link
```

💡 Tip: Hard link = same file, Soft link = shortcut.

86. Inode

- Inode stores metadata of a file.
- Contains: permissions, owner, size, timestamps, block pointers.

Example

```
ls -li file.txt # Show inode number
```

💡 Tip: Inode is like the file's identity.

87. Find Command

- Search files in directory hierarchy.
- Powerful with filters and actions.

Example

```
find / -name file.txt # Find by name
find /home -size +100M # Size > 100MB
```

💡 Tip: Find is very powerful.

88. Which Command

- Locate the path of an executable.

Example

```
which python # Path of python
which ls      # Path of ls
```

💡 Tip: which searches in \$PATH.

89. Whereis Command

- Searches binary, source and man page.

Example

```
whereis ls
# ls: /bin/ls /usr/share/man/man1/ls.1.gz
```

💡 Tip: Faster than find for binaries.

90. Type Command

- Tells how a command is interpreted.

Example

```
type ls # ls is alias /bin/ls
type cd # cd is a shell builtin
```

💡 Tip: Helps to know command type.



DAY 4 SUMMARY

- ✓ Understood Linux directory structure
- ✓ Learned file types and permissions
- ✓ Practiced ownership and group concepts
- ✓ Explored links, inodes and special bits

- ✓ Used essential file system commands
- ✓ Learned hidden files and default permissions
- ✓ Good progress - keep practicing daily!

**CONSISTENCY TODAY, EXPERTISE TOMORROW!**

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 04-JUNE-2025

PAGE 15 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



91. Mount vs Unmount

- Mount : Attach a filesystem to directory.
- Unmount : Detach the filesystem.

Example

```
mount /dev/sdb1 /mnt/data
umount /mnt/data
```

💡 Tip: Unmount before removing a device.

92. /etc/fstab

- Table of filesystems.
- Auto mount at boot.

Example

```
/dev/sdb1 /mnt/data ext4 defaults 0 2
↓         ↓         ↓         ↓         ↓
Device   Mount Point Type Options Dump
```

💡 Tip: 0 = no backup, 2 = check after root.

93. df Command (Disk Free)

- Shows disk space usage.

Example

```
df -h          # Human readable
df -Th         # Show filesystem type
```

💡 Tip: Use -h for easy reading.

94. du Command (Disk Usage)

- Show disk usage of files/directories.

Example

```
du -sh /home/user      # Summary
du -ah /var | less     # Detailed list
```

💡 Tip: Great for finding large files.

95. Hard Disk vs Partition

Type	Hard Disk	Partition
Definition	Physical device	Logical part
Can be divided	Yes	No
Example	/dev/sda	/dev/sda1
Size	Big	Smaller

💡 Tip: Partition is part of a Hard Disk.

96. Block Device vs Char Device

Type	Description	Example
Block	Data in blocks	/dev/sda
Character	Data as stream	/dev/tty

💡 Tip: Disks are block, terminals are char.

97. Special Directories

- /dev - Devices
- /proc - Process & kernel info
- /sys - System info
- /run - Runtime data
- /tmp - Temporary files

💡 Tip: These are important system dirs.

98. File System Types (Linux)

- ext4 - Default Linux FS
- xfs - High performance
- btrfs - Advanced features
- tmpfs - In-memory FS
- nfs - Network file system

💡 Tip: ext4 is most widely used.

99. Bad Blocks

- Damaged blocks on disk.
- Can cause data loss.

Check bad blocks

```
fsck -c /dev/sda1
```

💡 Tip: Always backup important data.

100. fsck Command

- Check and repair filesystem.

Example

```
fsck /dev/sda1      # Check & repair
fsck -n /dev/sda1   # Check only
```

💡 Tip: Run on unmounted filesystem.

101. mkfs Command

- Create a filesystem on partition.

Example

```
mkfs ext4 /dev/sdb1
mkfs xfs /dev/sdb1
```

💡 Tip: Erases all data on partition.

102. Resize Filesystem

- Grow or shrink filesystem.

Example

```
resize2fs /dev/sda1      # ext4
xfs_growfs /mnt/data     # xfs (grow only)
```

💡 Tip: Shrink is risky, always backup.

103. Copy File vs Move File

Command	Use	Example
cp	Copy File	cp a.txt b.txt
mv	Move or rename	mv a.txt /tmp/

💡 Tip: mv can be used to rename.

104. Soft Link (Symlink)

- Points to original file.
- If original deleted, link breaks.

Example

```
ln -s /home/user/file.txt link.txt
ls -l link.txt
```

💡 Tip: Like a shortcut.

105. Hard Link

- Points to same inode.
- Works within same filesystem.

Example

```
ln /home/user/file.txt hardlink.txt
ls -l hardlink.txt
```

💡 Tip: Hard link survives even if original removed.



DAY 4 SUMMARY

- ✓ Understood file system structure
- ✓ Practiced essential commands
- ✓ Learned permissions & links

- ✓ Explored filesystem types
- ✓ Used df, du, fsck, mkfs
- ✓ Gained strong Linux fundamentals

CONSISTENCY TODAY, EXPERTISE TOMORROW!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 04-JUNE-2025

PAGE 16 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



106. Find vs Locate

- find : Searches in real-time.
- locate : Uses updated database.

Example

```
find / -name file.txt # Slower
locate file.txt       # Faster
```

💡 Tip: locate is faster but may be outdated

107. Updatedb (for locate)

- Updates the locate database.
- Usually runs daily via cron.

Example

```
sudo updatedb
```

💡 Tip: Run manually after big changes.

108. Head vs Tail

- head : Show first lines.
- tail : Show last lines.

Example

```
head -n 10 file.txt # First 10 lines
tail -n 10 file.txt # Last 10 lines
tail -f log.txt     # Follow new lines
```

💡 Tip: tail -f is great for live logs.

109. Less vs More

- View file content page by page.
- less is more powerful.

Example

```
less file.txt
more file.txt
```

💡 Tip: Press 'q' to quit.

110. Cat vs Tac

- cat : Shows from top to bottom.
- tac : Shows from bottom to top.

Example

```
cat file.txt # Normal
tac file.txt # Reverse order
```

💡 Tip: tac = cat backward.

111. Grep Command

- Search text patterns in files.
- Very powerful with options.

Example

```
grep 'error' log.txt # Search 'error'
grep -i 'error' log.txt # Ignore case
grep -r 'error' /var/log # Recursive
grep -n 'error' log.txt # Show line no.
```

💡 Tip: Use regex for advanced search.

112. Grep with Options

Option	Use
-i	Ignore case
-r	Recursive search
-n	Show line number
-o	Show only matched text
-v	Invert match (not match)

💡 Tip: Combine options for power.

113. Redirect Output

- Save output to file.
- Overwrite or append.

Example

```
command > file.txt # Overwrite
command >> file.txt # Append
command 2> error.log # Error output
command &> all.log # Both output
```

💡 Tip: >> is safe, it won't erase old data.

114. Tee Command

- Save output and also show on screen.

Example

```
ls -l | tee list.txt # Save + display
cat file.txt | tee out.log # Save + display
```

💡 Tip: Great for logs and reports.

115. Pipes (|)

- Send output of one command to another.
- Build powerful command chains.

Example

```
ps aux | grep nginx
cat file.txt | grep error
df -h | sort -h
```

💡 Tip: Pipes save time and keystrokes.

116. Command Substitution

- Use output of a command inside another.

Example

```
echo "Today is $(date)"
mkdir backup_$(date +%F)
```

💡 Tip: \$() runs command inside.

117. Wildcards Recap

Wildcard	Meaning
*	Any number of characters
?	Any single character
[abc]	One character from set
[a-z]	Any letter a to z
[0-9]	Any digit

💡 Tip: Wildcards save time & effort.

118. Quoting in Linux

- Single quotes: Exact text.
- Double quotes: Allows variables.

Example

```
echo 'Today is $USER' # Exact text
echo "Hello $USER"    # Expands variable
```

💡 Tip: Use quotes wisely.

119. Escape Characters

- Escape special characters.
- Common escape: \

Example

```
echo "I love Linux\!" # Output: I love Linux!
echo "Line1\nline2"   # New Line
```

💡 Tip: \ is your escape key.

120. Summary of Key Takeaways

- ✓ Understand directory structure
- ✓ Work with files and directories
- ✓ Use permissions and ownership
- ✓ Search, filter and process with power
- ✓ Practice daily and become expert!



KEEP PRACTICING - YOU WILL MASTER LINUX!

DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 04-JUNE-2025

PAGE 18 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



136. Important Directories (Recap)

- `/bin` : Essential user commands
- `/sbin` : System admin commands
- `/etc` : Config files
- `/var` : Variable data (logs, spool)
- `/tmp` : Temporary files
- `/home` : User home directories
- `/usr` : User installed programs

💡 Tip: Know these dirs like the back of your hand.

137. Absolute vs Relative Path

- Absolute path: Starts from root (`/`)
- Relative path: Starts from current dir

Example

Absolute: `/home/user/docs/file.txt`
Relative: `docs/file.txt`
or
`../docs/file.txt` (go up one level)

💡 Tip: Use absolute path in scripts.

138. `..`, `..` and `~`

- `.` : Current directory
- `..` : Parent directory
- `~` : Home directory
- `/` : Root directory

Example

`cd .` # stay here
`cd ..` # go to parent
`cd ~` # go to home

💡 Tip: Very useful shortcuts.

139. Touch Command

- Create empty file.
- Update timestamp of existing file.

Example

`touch newfile.txt`
`touch file1.txt file2.txt`
`touch -r oldfile.txt newfile.txt`

💡 Tip: Creates file if not exists.

140. File Date & Time

- `atime` : last access time
- `mtime` : last modification time
- `ctime` : last change time

Example

`stat file.txt` # shows all times

💡 Tip: Useful for auditing & debugging.

141. File Watch Command

- Watch changes in real-time.
- Great for logs and debugging.

Example

`watch -n 2 ls -l`
`watch -n 1 df -h`

💡 Tip: Press `Ctrl + C` to stop.

142. Tree Command

- Display directory structure in tree.
- Very helpful to visualize.

Example

`tree`
`tree -L 2` # limit depth
`tree -a` # show hidden files

💡 Tip: Install if not available (auto opt install tree)

143. Disk Usage by Directory

- Find largest directories.
- Good for cleanup.

Example

`du -sh * | sort -h`
`du -ah /var | sort -rh | head -10`

💡 Tip: `-h` = human readable, `-s` = summary.

144. Find Large Files

- Find files bigger than given size.

Example

`find / -type f -size +100M`
`find /var -type f -size +50M`
`find . -type f -size +10M -exec ls -lh {} \;`

💡 Tip: Be careful with `/` (root search).

145. Count Files & Directories

- Count total files or directories.

Example

`find . -type f | wc -l` # files count
`find . -type d | wc -l` # dirs count
`ls -l | wc -l` # current dir count

💡 Tip: Quick way to get numbers.

146. File Content Preview

- View beginning of file.
- Useful for large files.

Example

`head file.txt` # first 10 lines
`head -20 file.txt` # first 20 lines
`tail file.txt` # last 10 lines
`tail -20 file.txt` # last 20 lines

💡 Tip: Use `-f` with tail for live logs.

147. Tail -f (Live Logs)

- Continuously watch new lines.
- Essential for log monitoring.

Example

`tail -f /var/log/syslog`
`tail -f /var/log/nginx/access.log`
`tail -f app.log`

💡 Tip: Press `Ctrl + C` to stop.

148. Redirection Basics

- Redirect output to file.
- Overwrite and append.

Example

`command > file.txt` # overwrite
`command >> file.txt` # append
`command 2> error.log` # error output
`command > out.log 2>&1` # both

💡 Tip: `>` overwrite, `>>` append.

149. Pipe (|) Command

- Take output of one command as input to another.

Example

`ls -l | grep txt`
`df -h | sort -hr`
`ps aux | grep nginx`

💡 Tip: Pipes make commands more powerful.

150. Permission Summary

Permission	Symbol	Octal	Meaning
Read	r	4	Read
Write	w	2	Write
Execute	x	1	Execute
No access	-	0	None

💡 Tip: `rwx` = 7, `rw-` = 6, `r-x` = 5, `---` = 4

DAY 4 SUMMARY



- ✓ Explored Linux file system structure
- ✓ Learned key commands and options
- ✓ Practiced permissions, links, and redirection

- ✓ Understood logs, disk usage, and monitoring
- ✓ Hands-on practice builds strong foundation
- ✓ Keep practicing daily - You'll master it!



TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



151. Hard Link vs Soft Link

- Hard Link : Same inode, multiple names.
- Soft Link (Symlink) : Pointer to path.

Example

```
ln file1 hardlink.txt # Hard Link
ln -s file1 symlink.txt # Soft Link
```

💡 Hard link survives if original removed.

152. Link Command

- Create hard links to files.
- Increases link count.

Example

```
ln file1 file2 # file2 is another
               # hard link to file1
```

💡 Both files share same data.

153. readlink Command

- Displays value of a symbolic link.
- Does not follow the link.

Example

```
readlink symlink.txt # Show target path
```

💡 Great for debugging links.

154. Realpath Command

- Shows absolute path, resolves symlinks.
- Cleans up path.

Example

```
realpath ../docs/./file.txt
realpath -s symlink.txt
```

💡 Use in scripts for safe paths.

155. ln -t and ln -T

- ln -t : Last argument is target directory
- ln -T : Treat target as normal file

Example

```
ln -t /backup file1 file2 # both to /backup
ln -T file1 backup_file1 # no dir treat
```

💡 Avoid mistakes with target.

156. umount vs eject

- umount : Safely detach filesystem.
- eject : For removable media.

Example

```
umount /mnt/usb
eject /dev/sdb
```

💡 Always unmount before removing.

157. System Directories (Recap)

- /etc - Config files
- /var - Variable data (logs, spool)
- /usr - User programs
- /bin - Essential binaries
- /sbin - System binaries

💡 Know where things live.

158. Temporary Directories

- /tmp - Temp files (cleared on reboot)
- /var/tmp - Temp files (may persist)

Example

```
ls -lh /tmp
ls -lh /var/tmp
```

💡 Don't store important data in /tmp.

159. Swap File

- Used when RAM is full.
- Helps system to not crash.

Example

```
free -h # Show swap usage
swapon --show # Active swap
```

💡 Swap = safety net, not performance.

160. du vs df (Recap)

Command	Purpose
du	Check file/dir size
df	Check disk space

💡 du = usage of files, df = space on FS.

161. ncddu Command

- Interactive disk usage analyzer.
- Better alternative to du.

Example

```
ncdu /home/user
```

💡 Install with: `sudo apt install ncddu`

162. locate Command

- Search files using database.
- Very fast search.

Example

```
locate filename.txt
updatedb # Update database
```

💡 Database updates daily via cron.

163. find -exec

- Execute a command on found files.
- Powerful in automation.

Example

```
find /logs -name "*.log" -exec rm {} \;
find /tmp -type f -exec ls -lh {} \;
```

💡 {} is placeholder for found file.

164. find -delete

- Delete files directly using find.
- Safer than `rm -rf`.

Example

```
find /tmp -name "*.log" -delete
```

💡 Use with care!

165. truncate Command

- Set file size to given value.
- Can clear or shrink file.

Example

```
truncate -s 0 file.txt # Empty file
truncate -s 1M file.txt # Set 1MB size
```

💡 Useful for log files.

166. stat Command

- Show detailed file metadata.
- Includes inode, permissions, time.

Example

```
stat file.txt
```

💡 Great for troubleshooting.

167. touch Command

- Create empty file or update timestamp.
- Useful in scripting.

Example

```
touch file.txt
touch -d "2025-06-04" file.txt
```

💡 -d sets custom date/time.

168. Summary of Day 4

- ✓ Explored file types and permissions
- ✓ Used links, find, grep, and mount
- ✓ Learned useful commands
- ✓ Understood real-world usage
- ✓ Practice daily and become expert!

PRACTICE TODAY - MASTER TOMORROW !



DAY 4 SUMMARY

- ✓ Covered important Linux file commands
- ✓ Worked with permissions, links, search
- ✓ Learned disk usage & mounting

- ✓ Practiced real-world scenarios
- ✓ Gained strong command-line skills
- ✓ Keep learning, keep growing!





TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



169. xargs Command

- Builds and executes command lines from standard input.
- Useful with find.

Example

```
find . -name "*.log" | xargs ls -l
find . -type f | xargs rm -f
```

Tip: xargs avoids "Argument list too long".

170. tee Command

- Reads from standard input and writes to file and stdout.

Example

```
ls -l | tee list.txt
cat file.txt | tee -a file.log
```

Tip: -a appends to file.

171. which vs whereis

- which: Shows full path of command.
- whereis: Shows binary, source, man pages.

Example

```
which ls # /bin/ls
whereis ls # ls: /bin/ls /usr/share/man/man1/ls.1.gz
```

Tip: which is for executable only.

172. type Command

- Tells how a command will be interpreted.
- Useful to know alias or builtin.

Example

```
type ls # alias or /bin/ls
type cd # builtin
```

Tip: Helps in troubleshooting.

173. alias Command

- Create shortcut for long commands.
- Saves time and keystrokes.

Example

```
alias ll='ls -aF'
alias k='kubectl'
```

Tip: Add to ~/.bashrc to make permanent.

174. unalias Command

- Removes an alias.
- Restores original command.

Example

```
unalias ll
unalias k
```

Tip: Keeps shell clean.

175. History Command

- Shows list of previously executed commands.
- Useful for reuse.

Example

```
history # list history
history | grep docker # filter
!100 # run command no 100
```

Tip: Use ↑ key to scroll history.

176. Jobs Command

- Shows background jobs.
- Helps manage running tasks.

Example

```
jobs # list jobs
jobs -l # with PID
fg %1 # bring job 1 to foreground
```

Tip: Ctrl+Z to pause, bg to resume.

177. kill Command

- Sends signal to a process.
- Used to stop or terminate.

Example

```
ps aux | grep nginx
kill 1234 # send TERM
kill -9 1234 # force kill
```

Tip: Prefer TERM before KILL.

178. ps Command

- Show running processes.
- Essential for monitoring.

Example

```
ps aux # all processes
ps -ef | grep java # filter process
ps -u user # by user
```

Tip: Use top/htop for live view.

179. top / htop

- Real-time process, CPU, memory view.
- htop is user-friendly.

Example

```
top # in-built
htop # better visualization
```

Tip: Press q to quit top/htop.

180. df vs du (Recap)

Command	Purpose
df	Disk space usage
du	Directory space usage

Tip: df = filesystem, du = files/dirs.

181. file Command

- Determines file type.
- Useful for unknown files.

Example

```
file image.png # PNG image data
file script.sh # Bourne shell script
```

Tip: Useful in forensics & scripting.

182. gzip / gunzip

- Compress and decompress files.
- Saves disk space.

Example

```
gzip file.txt # compress
gunzip file.txt.gz # decompress
```

Tip: .gz extension after compression.

183. tar Command

- Archive multiple files together.
- Commonly used with compression.

Example

```
tar -cvf backup.tar /home/user
tar -xvf backup.tar
tar -czvf backup.tar.gz /home/user
```

Tip: c=create, x=extract, v=verbose.



DAY 4 SUMMARY

- ✓ Explored important commands for files, links, permissions, search, and process management.
- ✓ Learned shortcuts and real-time monitoring tools.
- ✓ Practiced archiving, compression and usage analysis.

- ✓ Strong foundation in Linux file system.
- ✓ Daily practice makes you confident.
- ✓ Keep learning, keep growing!



DEVOPS 60 DAYS

ZERO TO HERO SERIES

DAY 4

DATE : 04-JUNE-2025

PAGE 17 OF 25

TOPIC : LINUX FILE SYSTEM BASICS

Small steps daily.

Big skills for Life!

Keep Practicing !



121. File Ownership (chown)

- Change owner and/or group of a file.
- Format: `chown [user][:group] file`

Example

```
chown sachin report.txt    # Change owner
chown :devops /data/       # Change group
chown sachin:devops file.txt # Both
```

💡 Tip: Only root or file owner can chown.

122. File Permissions (chmod numeric)

- Numeric permissions: user group others
- Read=4, Write=2, Execute=1
- Sum the values for the permission

Example

```
chmod 755 script.sh    # rwxr-xr-x
chmod 644 file.txt     # rw-r--r--
```

💡 Tip: Use symbolic chmod for readability.

123. umask

- Default permissions mask for new files.
- Subtracts from permission 666 (files)
- Subtracts from 777 (directories)

Example

```
umask 022    # New Files: 644 Dirs: 755
umask 002    # New Files: 664 Dirs: 775
```

💡 Tip: Set in ~/.bashrc for permanent use.

124. Special Files

- Device files represent hardware.
- Types: block, character, FIFO, socket

Example

```
ls -l /dev/sda1    # Block device
ls -l /dev/tty     # Character device
ls -l /tmp/mypipe  # FIFO (named pipe)
```

💡 Tip: Use `ls -l` to see file type (first char).

125. Mount Options

- Control how a file system is mounted.
- Common options: `ro`, `rw`, `noexec`, `nosuid`

Example

```
mount -o ro /dev/sdb1 /mnt/backup
mount -o rw,noexec /dev/sdb1 /mnt/data
```

💡 Tip: Use /etc/fstab for permanent mounts.

126. /etc/fstab Explained

- Defines file systems to mount at boot.
- Format: device mount_point fs_type options

Example

```
/dev/sdb1 /data ext4 defaults 0 2
UUID=xxxx /backup ext4 ro,nofail 0 2
```

💡 Tip: Test with `mount -a` before reboot.

127. File System Types (Common)

- ext4: Linux default, journaling FS
- xfs: High performance
- btrfs: Advanced features, snapshots
- tmpfs: In-memory file system

Example

```
df -T    # See file system types
```

💡 Tip: Choose FS based on workload needs.

128. Hard Link vs Symbolic Link

- Hard link: Same inode, cannot cross FS
- Symbolic link: Points to another file/path

Example

```
ln file1 hardlink.txt    # Hard link
ln -s /var/log syslog    # Symbolic link
```

💡 Tip: Symlink is like a shortcut.

129. Inode

- Inode stores metadata about a file.
- Contains: permissions, owner, size, links, timestamps, block pointers

Example

```
ls -li file.txt    # Shows inode number
```

💡 Tip: Many small files can use many inodes.

130. File Size

- Use `ls`, `du`, `stat` to check file size.
- Units: B, KB, MB, GB, TB

Example

```
ls -lh file.txt    # Human readable size
du -sh /var/log    # Directory size
stat file.txt      # Detailed info
```

💡 Tip: Use `-h` for human readable output.

131. File Search

- Find files based on name or pattern.
- Use `find` command

Example

```
find /home -name "*.txt"
find / -type f -size +100M
```

💡 Tip: Use `-iname` for case-insensitive search.

132. Compression & Archiving

- Compress files and directories.
- Common tools: `tar`, `gzip`, `zip`

Example

```
tar -czf backup.tar.gz /home/user
zip -r logs.zip /var/log
```

💡 Tip: `tar` keeps permissions and structure.

133. File Recovery

- Recover deleted files (if not overwritten).
- Use `extundelete` or `testdisk`

Example

```
extundelete /dev/sda1 --restore-all
testdisk /dev/sda1
```

💡 Tip: Act fast. Data may be overwritten.

134. Best Practices

- Use proper permissions
- Regular backups
- Monitor disk usage
- Clean up unused files

💡 Tip: Small steps today, big results tomorrow.

135. Quick Commands Recap

<code>ls</code>	# List files
<code>cd</code>	# Change directory
<code>pwd</code>	# Current directory
<code>mkdir</code>	# Make directory
<code>rm</code>	# Remove file/directory
<code>cp</code>	# Copy
<code>mv</code>	# Move or rename
<code>chmod</code>	# Change permissions
<code>chown</code>	# Change owner/group
<code>df</code>	# Disk usage
<code>du</code>	# Directory size



DAY 4 SUMMARY

- ✓ Learned about files, permissions, links
- ✓ Understood inodes, mounts, and file systems
- ✓ Practiced essential Linux commands
- ✓ Ready for real-world DevOps tasks

- ✓ Practice daily
- ✓ Explore with hands-on
- ✓ Stay curious and keep building

KEEP LEARNING - KEEP BUILDING - KEEP GROWING !

